

market.c

```
?#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "types.h"
#include "functions.h"

void initMarket(GameState game[])
{
    int i;

    for(i = 0; i < NO_GROUP; i++)
        game[0].economy.groupCooldownUntilRound[i] = 0;

    game[0].economy.lastBoomGroup = -1;
    game[0].economy.lastDeclineGroup = -1;
}

void groupName(PropertyGroup group, char buffer[])
{
    if(group == BROWN)
        strcpy(buffer, "Brown");
    else if(group == LIGHT_BLUE)
        strcpy(buffer, "Light Blue");
    else if(group == PINK)
        strcpy(buffer, "Pink");
    else if(group == ORANGE)
        strcpy(buffer, "Orange");
    else if(group == RED)
        strcpy(buffer, "Red");
    else if(group == YELLOW)
        strcpy(buffer, "Yellow");
    else if(group == GREEN)
        strcpy(buffer, "Green");
    else if(group == DARK_BLUE)
        strcpy(buffer, "Dark Blue");
    else
        strcpy(buffer, "Unknown");
}

/* Picks a random group that isn't on cooldown and isn't one of the
   two groups to avoid (Rule-LK 30, 33). */
PropertyGroup pickEligibleGroup(GameState game[], int currentRound, int avoid1, int avoid2)
{
    int candidates[NO_GROUP];
    int count;
    int g;

    count = 0;

    for(g = 0; g < NO_GROUP; g++)
    {
        if(g == avoid1 || g == avoid2)
            continue;

        if(game[0].economy.groupCooldownUntilRound[g] > currentRound)
            continue;

        candidates[count] = g;
        count++;
    }
}
```

```

    if(count == 0)
        return (PropertyGroup)((avoid1 + 1) % NO_GROUP);    /* rare fallback */

    return (PropertyGroup)candidates[rand() % count];
}

/* Rule-LK 30-34: every 10 rounds, one group booms and a different
one declines, each for 10 rounds. All effects are temporary
modifiers (Rule-LK 34) so no base value or cost is ever changed. */
void reviewPropertyMarket(GameState game[], int currentRound)
{
    PropertyGroup boomGroup;
    PropertyGroup declineGroup;
    char boomName[12];
    char declineName[12];

    boomGroup = pickEligibleGroup(game, currentRound, game[0].economy.lastBoomGroup, -1);
    declineGroup = pickEligibleGroup(game, currentRound, game[0].economy.lastDeclineGroup, boomGroup);

    groupName(boomGroup, boomName);
    groupName(declineGroup, declineName);

    printf("\n=== Property Market Review ===\n");
    printf("Market Boom : %s (values +20%%, rent +25%%) for 10 rounds.\n", boomName);
    printf("Market Decline : %s (values -15%%, rent -20%%) for 10 rounds.\n", declineName);

    addSourcedModifier(game, MOD_GROUP_VALUE, boomGroup, -1, 120, 10, SRC_BOOM);
    addSourcedModifier(game, MOD_GROUP_RENT, boomGroup, -1, 125, 10, SRC_BOOM);

    addSourcedModifier(game, MOD_GROUP_VALUE, declineGroup, -1, 85, 10, SRC_DECLINE);
    addSourcedModifier(game, MOD_GROUP_RENT, declineGroup, -1, 80, 10, SRC_DECLINE);

    /* Rule-LK 31/32/34: the boom group's mortgage values rise by 15%,
    direct purchase prices rise by 15% and construction costs rise
    by 10%; the decline group's mortgage values fall by 10% and
    auction starting prices fall by 25%. All are scoped to the
    affected group. */
    addSourcedModifier(game, MOD_MARKET_MORTGAGE, boomGroup, -1, 115, 10, SRC_BOOM);
    addSourcedModifier(game, MOD_PURCHASE_PRICE, boomGroup, -1, 115, 10, SRC_BOOM);
    addSourcedModifier(game, MOD_CONSTRUCTION, boomGroup, -1, 110, 10, SRC_BOOM);

    addSourcedModifier(game, MOD_MARKET_MORTGAGE, declineGroup, -1, 90, 10, SRC_DECLINE);
    addSourcedModifier(game, MOD_AUCTION_PRICE, declineGroup, -1, 75, 10, SRC_DECLINE);

    game[0].economy.groupCooldownUntilRound[boomGroup] = currentRound + MARKET_COOLDOWN_ROUNDS;
    game[0].economy.groupCooldownUntilRound[declineGroup] = currentRound + MARKET_COOLDOWN_ROUNDS;

    game[0].economy.lastBoomGroup = boomGroup;
    game[0].economy.lastDeclineGroup = declineGroup;
}

/* Table 4: one card every 15 rounds, active for 15 rounds. */
void drawRegionalCard(GameState game[])
{
    int choice;

    choice = rand() % REGIONAL_CARD_COUNT;

    printf("\n=== Regional Development Card ===\n");

    switch(choice)
    {

```

```

case 0:
    printf("Southern Tourism Boom : Galle Fort, Unawatuna and "
           "Hikkaduwa rental income +40%%.\n");
    addSourcedModifier(game, MOD_GROUP_RENT, YELLOW, -1, 140, 15, SRC_REGIONAL);
    break;

case 1:
    printf("Port City Expansion : Pettah and Maradana values +25%%.\n");
    addSourcedModifier(game, MOD_GROUP_VALUE, BROWN, -1, 125, 15, SRC_REGIONAL);
    addSourcedModifier(game, MOD_INDEX_VALUE, -1, 5, 125, 15, SRC_REGIONAL); /* Colombo Fort Station
    break;

case 2:
    printf("IT Industry Growth : Maharagama, Nugegoda and "
           "Kottawa values +20%%.\n");
    addSourcedModifier(game, MOD_GROUP_VALUE, PINK, -1, 120, 15, SRC_REGIONAL);
    break;

case 3:
    printf("Northern Development Programme : Jaffna Town, Nallur "
           "and Trincomalee values +30%%.\n");
    addSourcedModifier(game, MOD_GROUP_VALUE, GREEN, -1, 130, 15, SRC_REGIONAL);
    break;

case 4:
    printf("Tea Export Boom : Nuwara Eliya value +35%%.\n");
    addSourcedModifier(game, MOD_INDEX_VALUE, -1, 37, 135, 15, SRC_REGIONAL);
    break;

case 5:
    printf("Airport Expansion : Negombo, Katunayake and "
           "Ja-Ela rents +30%%.\n");
    addSourcedModifier(game, MOD_GROUP_RENT, ORANGE, -1, 130, 15, SRC_REGIONAL);
    break;

case 6:
    printf("University City Growth : Peradeniya and "
           "Kandy City values +20%%.\n");
    addSourcedModifier(game, MOD_GROUP_VALUE, RED, -1, 120, 15, SRC_REGIONAL);
    break;

case 7:
    printf("Beach Pollution : Southern coastal rents -30%%.\n");
    addSourcedModifier(game, MOD_GROUP_RENT, YELLOW, -1, 70, 15, SRC_REGIONAL);
    break;

case 8:
    printf("Flood Damage : Low-lying coastal properties lose 20%% value.\n");
    addSourcedModifier(game, MOD_GROUP_VALUE, YELLOW, -1, 80, 15, SRC_REGIONAL);
    break;

case 9:
    printf("Transport Strike : Railway revenue reduced by 40%%.\n");
    addSourcedModifier(game, MOD_RAIL_RENT, -1, -1, 60, 15, SRC_REGIONAL);
    break;

case 10:
    printf("Electricity Tariff Increase : Utility rent +25%%.\n");
    addSourcedModifier(game, MOD_UTIL_RENT, -1, -1, 125, 15, SRC_REGIONAL);
    break;

case 11:
    printf("Water Shortage : Utility revenue +20%%.\n");

```

```

        addSourcedModifier(game, MOD_UTIL_RENT, -1, -1, 120, 15, SRC_REGIONAL);
        addSourcedModifier(game, MOD_INDEX_VALUE, -1, 13, 90, 15, SRC_REGIONAL);
        addSourcedModifier(game, MOD_INDEX_VALUE, -1, 29, 90, 15, SRC_REGIONAL);
        break;

    default:
        break;
}
}

/* Prints a section title followed by a separator line. */
void printSectionHeader(char title[])
{
    printf("%s\n", title);
    printf("-----\n");
}

/* Prints a Market Boom or Market Decline block. */
void printBoomDecline(char title[], char groupNameBuffer[], int percent, int roundsLeft)
{
    printSectionHeader(title);
    printf("%s (%+d%)\n", groupNameBuffer, percent - 100);
    printf("Rounds Remaining : %d\n", roundsLeft);
}

/* Writes one Regional Development card line into out. Returns 1 if
the modifier has a line to show, 0 otherwise. */
int describeRegionalCondition(GameState game[], ActiveModifier mod, char out[])
{
    char name[48];

    if(mod.type == MOD_GROUP_VALUE || mod.type == MOD_GROUP_RENT)
    {
        if(mod.group >= 0 && mod.group < NO_GROUP)
            groupName((PropertyGroup)mod.group, name);
        else
            strcpy(name, "Unknown");

        if(mod.type == MOD_GROUP_VALUE)
            sprintf(out, "%s (%+d%)", name, mod.percent - 100);
        else
            sprintf(out, "%s rents (%+d%)", name, mod.percent - 100);
    }
    else if(mod.type == MOD_INDEX_VALUE)
    {
        sprintf(out, "%s (%+d%)",
            game[0].board[mod.index].name, mod.percent - 100);
    }
    else if(mod.type == MOD_RAIL_RENT)
    {
        sprintf(out, "Railway rents (%+d%)", mod.percent - 100);
    }
    else if(mod.type == MOD_UTIL_RENT)
    {
        sprintf(out, "Utility rents (%+d%)", mod.percent - 100);
    }
    else
    {
        return 0;
    }

    return 1;
}

```

```

/* Prints the "Regional Development" section. Returns 1 if at least
one card effect was shown. */
int printRegionalSection(GameState game[])
{
    int i;
    int shown;

    shown = 0;

    for(i = 0; i < game[0].economy.modifierCount; i++)
    {
        ActiveModifier mod;
        char line[100];

        mod = game[0].economy.modifiers[i];

        if(mod.source != SRC_REGIONAL)
            continue;

        if(!describeRegionalCondition(game, mod, line))
            continue;

        if(!shown)
        {
            printSectionHeader("Regional Development");
            shown = 1;
        }

        printf("%s\n", line);
        printf("Rounds Remaining : %d\n", mod.roundsLeft);
    }

    return shown;
}

/* Writes one "Other Active Conditions" line into out. Returns 1 if
the modifier has a line to show, 0 otherwise. */
int describeOtherCondition(GameState game[], ActiveModifier mod, char out[])
{
    char name[32];

    if(mod.type == MOD_GROUP_VALUE || mod.type == MOD_GROUP_RENT)
    {
        groupName((PropertyGroup)mod.group, name);

        if(mod.type == MOD_GROUP_VALUE)
            sprintf(out, "%s : values x%d%% (%d rounds remaining)",
                    name, mod.percent, mod.roundsLeft);
        else
            sprintf(out, "%s : rents x%d%% (%d rounds remaining)",
                    name, mod.percent, mod.roundsLeft);
    }
    else if(mod.type == MOD_INDEX_VALUE)
    {
        sprintf(out, "%s : value x%d%% (%d rounds remaining)",
                game[0].board[mod.index].name, mod.percent, mod.roundsLeft);
    }
    else if(mod.type == MOD_RAIL_VALUE)
    {
        sprintf(out, "Railway values : x%d%% (%d rounds remaining)",
                mod.percent, mod.roundsLeft);
    }
}

```

```

else if(mod.type == MOD_RAIL_RENT)
{
    sprintf(out, "Railway rents : x%d%% (%d rounds remaining)",
        mod.percent, mod.roundsLeft);
}
else if(mod.type == MOD_UTIL_RENT)
{
    sprintf(out, "Utility rents : x%d%% (%d rounds remaining)",
        mod.percent, mod.roundsLeft);
}
else
{
    return 0;
}

return 1;
}

```

/* Prints the "Other Active Conditions" section (events and government regulations still running). */

```

void printOtherConditions(GameState game[])
{
    int i;
    int shown;

    shown = 0;

    for(i = 0; i < game[0].economy.modifierCount; i++)
    {
        ActiveModifier mod;
        char line[100];

        mod = game[0].economy.modifiers[i];

        if(mod.source != SRC_GENERAL)
            continue;

        if(!describeOtherCondition(game, mod, line))
            continue;

        if(!shown)
        {
            printSectionHeader("Other Active Conditions");
            shown = 1;
        }

        printf("%s\n", line);
    }
}

```

/* Rule-LK 36: shows the currently active market conditions at the end of every round, in the Section 5 format. */

```

void displayMarketConditions(GameState game[])
{
    int i;
    int shown;

    printf("\n=====\\n");
    printf("Current Market Conditions\\n");
    printf("=====\\n");

    shown = 0;
}

```

```

/* Market Boom (Rule-LK 31) */
for(i = 0; i < game[0].economy.modifierCount; i++)
{
    char name[32];

    if(game[0].economy.modifiers[i].source != SRC_BOOM ||
       game[0].economy.modifiers[i].type != MOD_GROUP_VALUE)
        continue;

    groupName((PropertyGroup)game[0].economy.modifiers[i].group, name);
    printBoomDecline("Market Boom", name,
                     game[0].economy.modifiers[i].percent,
                     game[0].economy.modifiers[i].roundsLeft);

    shown = 1;
    break;
}

/* Market Decline (Rule-LK 32) */
for(i = 0; i < game[0].economy.modifierCount; i++)
{
    char name[32];

    if(game[0].economy.modifiers[i].source != SRC_DECLINE ||
       game[0].economy.modifiers[i].type != MOD_GROUP_VALUE)
        continue;

    groupName((PropertyGroup)game[0].economy.modifiers[i].group, name);
    printBoomDecline("Market Decline", name,
                     game[0].economy.modifiers[i].percent,
                     game[0].economy.modifiers[i].roundsLeft);

    shown = 1;
    break;
}

/* Regional Development Card effects (Table 4) */
if(printRegionalSection(game))
    shown = 1;

if(!shown)
    printf("No active market booms, declines or regional conditions right now.\n");

/* Other active event/regulation effects */
printOtherConditions(game);

printSectionHeader("Inflation");
printf("%+d%%\n", game[0].economy.inflationRate);

printSectionHeader("Current Loan Interest");
printf("%d%%\n", game[0].economy.loanInterestRate);

printSectionHeader("Current Income Tax Rate");
printf("%d%%\n", game[0].economy.incomeTaxRate);

printf("=====\n");
}

```